Predicting Basketball Player Positions from Seasonal Game Statistics

CS-C3240 – Machine Learning

1. Introduction

In basketball, players are usually assigned a playing position, which mostly reflects their offensive role. As there are five players on court, the most intuitive way would be to have five positions which has been the common way for a long time. Additionally, each position requires different physical attributes. For example, height and mass are an advantage under the basket, while speed is needed when bringing the ball up the court. However, the game is changing and favors versatility over specialization, leading to "positionless basketball" where players have roles rather than strict positions. The traditional positions, Point Guard, Shooting Guard, Small Forward, Power Forward and Center, can be compressed to Guard, Forward and Center for simplicity and to give more space for role-based thinking. Moreover, even these three classifications may not hold up anymore because of the desired versatility, for example Centers can initiate plays to create advantage when it is normally Guards' job.

This project aims to test if the three player positions can still be concluded from players' game statistics, i.e. their recorded actions on court without factoring in their height. The application domain is professional basketball analytics, specifically examining how modern, increasingly positionless playing styles are reflected in seasonal box score statistics. In this report, Section 2 formulates the machine learning problem. Section 3 details the data, feature selection, and classification methods. Section 4 presents and compares the results of the models, and Section 5 discusses the conclusions, limitations, and future directions of the project.

2. Problem formulation

The task is formulated as a supervised multiclass classification, where each data point represents one player's single-season performance as a multidimensional feature vector. The goal is to train a classifier to predict a player's position (the target label, see Section 3.2.) based on the performance statistics.

3. Methods

3.1. Data and Preprocessing

The project uses EuroLeague data as it is less isolation-heavy (i.e., less reliant on individual 1-on-1 plays) and more team-focused than NBA playing style. The data was scraped from the public API of EuroLeague's website using a Python script (see Appendix A). The dataset includes five consecutive seasons from season 2021-2022 through 2025-2026 (EuroLeague Basketball, 2026), giving us 973 data points to work with.

The raw data was parsed and turned into a `.csv` with the desirable columns, later processed into features. Each data point represents one player's seasonal statistics, i.e. the average of each game they played during a season. If a player has played for more than one season, each season

is a different data point. The statistics that are not percentages are normalized for full playing time (40 minutes) to eliminate the effects of different playing times, which are rather a sign of trust from the coach or being a better player than someone else rather than a feature we should use.

Data points with less than ten games played during the season or an average playing time of under ten minutes were dropped from the dataset as those are the most likely outliers, such as end-of-bench players and coaches. This was around 5 % of the dataset, decreasing the number of data points to 927.

3.2. Feature selection and labels

Feature selection and engineering were done based on domain knowledge. 9 features ended up being selected.

Table 1: Features

Feature and data type	**Selection reasoning**
3P Attempt Rate (3PAr), continuous $[0, 1]$	The ratio of 3P shots to all the in-game shots. If high, the shot selection leans to more shots taken further away from the basket, which indicates a Guard. Calculated 3PA/(2PA+3PA). Strong negative correlation with 2P noted (see Appendix B, Pearson Correlation Heatmap), but as this feature captures shot distribution rather than volume, it was retained.
3Pr, continuous $[0, 1]$	3-pointer shooting accuracy. Normally high among Guards and some shooting Forwards as they play further away from the basket and therefore need to have a decent shot from the 3P line.
2Pr, continuous $[0, 1]$	2-pointer shooting accuracy. Higher among Centers who play close to the basket and thus 2-pointers are high-percentage shots for them. The accuracy is lower when the shots are taken from mid-range or contested drives to the basket, i.e. among Forwards and Guards.
2P Made per 40min, continuous 0^+	Volume of 2P shooting. A high number of made 2-pointers is typical of Centers who often take a majority of their shots from close to the basket. Despite a negative correlation with 3PAr, the feature was retained for being about volume and not distribution.
Fouls Drawn (FD) per 40min, continuous 0^+	The number of times a player was fouled during a game. Often a sign of being active with the ball and physicality, which are often associated with Centers and driving Guards. This feature was abandoned after correlation analysis revealed high redundancy with FTr ($r = 0.72$).

Continued on next page

Table 1 – continued from previous page

Feature and data type	Selection reasoning
Free Throw rate (FTr) (FT Attempted/Field Goals Attempted), continuous $[0, 1]$	The frequency of drawing contact compared to shots taken in a game. Often high among Centers with the same reasoning as above. Not abandoned after noted correlation with FD as FTr normalizes drawing contact against total number of attempted shots, hence providing more reliable efficiency metric (Zakarian, 2026).
Assists (AST) per 40min, continuous 0^+	The number of baskets the player assisted, i.e. gave the last pass before a made shot. A high amount of assists is a sign of playmaking activity. Primarily high among Guards who are responsible for starting the play, however, some Forwards share the playmaking duties.
Total Rebounds (TR) per 40min, continuous 0^+	Traditional sign of a Center is high number of rebounds due to their interior position and physicality.
Blocks (BLK) per 40min, continuous 0^+	The number of times a player blocked a shot during a game. A sign of rim protection, and while the volumes are small, normally they are relatively higher among Centers.
AST/Turnovers (TO), continuous 0^+	The ratio of assists and turnovers (losses of ball possession). Ratio above 1.5-2.0 is frequent among good Guards who pass the ball a lot but do not lose it too many times. Forwards and Centers can often commit turnovers while making fewer assists. Main reason for this feature is the differentiation between a playmaking Guard and a Forward.

Note: While the table uses the phrase "during a game", the project uses seasonal averages (see Section 3.1.)

A Pearson correlation matrix was made to examine the features' correlations and find redundant features, resulting in abandoning one feature. For feature correlation heatmap, see Appendix B, section 'Pearson Correlation Heatmap'.

The labels are straight from EuroLeague's data where the teams have used the three-position classification for the players: Guard (G), Forward (F) and Center (C). Additionally, it is possible to use a five-position classification in basketball, where Guard and Forward are split into two subclasses, but these sub-roles are increasingly considered artificial and thus differentiating them would be even more demanding and cause more noise. Therefore, as EuroLeague itself has abandoned the five-position system, the labels used in the project are the three official roster designations. The positions create a 3-class target variable $y \in \{\mathrm{G}, \mathrm{F}, \mathrm{C}\}$. With each player-season being represented as a feature vector $x \in \mathbb{R}^9$, the task is learning a classifier $f : \mathbb{R}^9 \to \{\mathrm{G}, \mathrm{F}, \mathrm{C}\}$.

Table 2: Target labels explanations

Position/Role	Usual attributes and responsibilities in basketball
Guard (G)	Bringing the ball up the court, controlling game tempo and initiating plays, scoring and shooting long-range shots. Short, fast players with a decent shooting average from afar.
Forward (F)	Scoring from close and far away, off-ball movement, some rebounding. Average height; can be on the smaller side; rather fast and physical.
Center (C)	Rim protection, interior scoring.

3.3. Model and hypothesis space

The hypothesis space of a random forest consists of collections of tree-structured classifiers whose predictions are chosen by majority vote. Each tree splits the data at every node by comparing a feature to a learned threshold, giving non-linear decision boundaries. A random forest classifier was selected as the first classification method as basketball statistics were expected to have non-linear interactions with player roles. For example, a high number of rebounds alone does not signify a Center, but when the same player also has high volume on 2P and a lot of blocks, the likelihood of them being a Center increases considerably. Therefore, the positions can be characterized by feature combination thresholds (Jung, 2022). While a single tree would easily overfit, combining multiple random trees keeps the predictions more stable. Additionally, as the trees compare values relative to the threshold, scale standardization, for example between percentage and per-40-minutes statistics, is redundant. Class imbalances, however, needed to be considered by balancing the class weights. Gini impurity was chosen to be used in the random forest splits as its loss function as it performs similarly to information gain while being computationally cheaper (Jung, 2022). Misclassification error is overly simplistic as it only reacts to purity improvements that change the class prediction, missing small purity improvements.

The hypothesis space of multinomial logistic regression (MLR) consists of linear maps from the features to class scores (Jung, 2022). The softmax function turns these into class probabilities and gives linear decision boundaries. MLR was chosen for additional analysis as its coefficients provide a more detailed view into the relationships between features and classes. To use it, however, the data needs to be standardized. As with the random forest, the classes need balancing here as well. Log loss (cross-entropy) is used as the loss function as it is derived from maximum likelihood estimation, meaning that it produces probability estimates, not just classifications, and this makes the coefficients meaningful for analysis. It is also convex and therefore easy to optimize. Alternative loss functions, such as focal loss, address problems that are not present in this project, such as extreme class imbalance, and hinge loss would not produce probability outputs.

3.4. Splitting the Data

The data was split into a 20 % test set with 186 data points and an 80 % training set with 741 data points. The split reserves enough data for the training set while leaving enough for the test so that it can reliably estimate the performance. Since the same player may have played in multiple seasons, it was important that different seasons from one player do not end up on both sides of the split. Another consideration was that the classes were imbalanced as there were almost twice as many Forwards and Guards as there were Centers. To account for these, the data was grouped by player, and the class proportions in the subsets were kept the same as in the original dataset.

3.5. Training and Validation

For both models, training and parameter tuning used 5-fold cross-validation on the training set. Five folds were chosen as each training set (about 593 data points) is close to the training set size and each validation set (about 148 data points) is large enough. The computational cost of the grid search also stays manageable. Similarly to when splitting the data into the training and test set, class imbalances and multiple data points per player needed to be taken into account here as well. For each combination of chosen candidate values for hyperparameters, the training set was split into 5 folds using the same grouped, stratified strategy described above. To make parameter tuning more efficient, two passes were made: a general search and a specified search near the result parameters of the first pass. Macro-averaged F1 score (macro-F1) was chosen as the evaluation metric since it works well for datasets with imbalanced and equally important classes (Jung, 2022).

For logistic regression, the data was additionally standardized in each fold based on the training set's values, so that none of the scaling depends on the validation set's values. If the scaling were to happen outside of the folds, the values in the validation data would affect the scaling of the training data, essentially leaking information.

4. Results

The models were evaluated using 5-fold cross-validation on the training set to estimate generalization and tune parameters, with the macro-averaged F1 score serving as the primary evaluation. Random forest achieved a training macro-F1 of 0.820 and a validation macro-F1 of 0.787, indicating very mild overfitting, whereas multinomial logistic regression had very similar training and validation macro-F1 scores (0.794 and 0.792). As the models achieve very similar results, greater interpretability makes MLR the final choice. Both models generalize well to unseen data, with random forest and MLR having macro-F1 scores of 0.779 and 0.817 on test sets, respectively. MLR scored higher than random forest largely due to classifying Centers and Forwards better (see Appendix C), leading to higher overall test performance. The test error of MLR was 17.2 %, so around 32 of 186 data points were misclassified.

Based on the confusion matrices (Appendix C), both models' Guard predictions were the most accurate, while Forwards were confused the most with both Guards and Centers. The latter makes sense as Forwards are hybrid players, sharing responsibilities with both positions (see Section 3.2., Table 2). Neither model ever directly confuses Guards and Centers (0-1 % misclassifications), confirming that interior and perimeter playing skill sets remain distinct.

When comparing MLR coefficients heatmap and the random forest feature importances table (see Appendix C), both highlight the same three features: Total Rebounds (TR), 3P Attempt Rate (3PAr) and Assists (AST). However, while feature importance indicates only how much the feature influences split decisions, the regression coefficients add directional role polarization meaning that a feature, like high TR, can favor a label while penalizing another. Furthermore, both models agree on 3-point shooting accuracy (3Pr) being virtually redundant for classification.

5. Conclusions

This project indicates that the three basketball playing positions can be predicted with about 83 % accuracy from in-game seasonal statistics alone, without factoring in physical attributes, such as player height. The results show that multinomial logistic regression identifies players' roster positions, suggesting that recorded actions on the court remain reflective of their functional roles in recent EuroLeague seasons. However, missing features and inconsistent labels

may limit the results, and the test set contains only 186 data points from one split, making test estimates uncertain. These limitations slightly decrease the strength of conclusions that can be drawn.

The results were not optimal, as around 17 % of player-seasons were misclassified. The test errors primarily concerned the Forward position, which is likely explained by its role as an intermediate category between Guards and Centers. To address the positional overlap, future work could explore unsupervised learning methods, such as clustering. Instead of forcing players into predefined discrete positions, clustering could identify data-driven player types based on playing style similarities. This could reveal completely novel positions beyond the traditional three or five positions, providing valuable insights into team building, scouting and individual coaching. Moreover, incorporating player-tracking data or shot charts alongside traditional box score statistics could provide better context for capturing these hybrid roles.

Declaration of AI Use

Large language models (Google Gemini and Anthropic's Claude) were used for explaining concepts, grammar checks, vocabulary enhancement, and refining the report structure. Claude was additionally used to make sense of code documentation (e.g. pandas and sklearn). All analyses, domain rationales and conclusions remain original work.

References

EuroLeague Basketball. (2026). *EuroLeague Official Player Statistics (2021–2026 Seasons).* https://www.euroleaguebasketball.net/en/euroleague/stats/players/

Jung, A. (2022). *Machine Learning: The Basics.* Springer.

Zakarian, C. B. (2026). *The Four Factors That Decide Basketball Games.* NBA Analytic. https://nbaanalytic.com/articles/four-factors.html

A. data_scraper.py

```
import requests
import json
import pandas as pd


def request_teams_api(season):
    """
    takes a season year as a parameter, e.g. 2021 for the season 2021-22
    returns a list of team codes, e.g. IST
    """
    url = f"https://feeds.incrowdsports.com/provider/euroleague-feeds/v2/competitions
        /E/seasons/E{season}/clubs"
    payload = {}
    headers = {
        'accept': 'application/json, text/plain, */*',
        'dnt': '1',
        'accept-language': 'fi-FI,fi;q=0.9,en-US;q=0.8,en;q=0.7',
        'origin': 'https://www.euroleaguebasketball.net',
        'priority': 'u=1, i',
        'referer': 'https://www.euroleaguebasketball.net/',
        'sec-ch-ua': '"Chromium";v="152", "Not?A_Brand";v="24", "Google Chrome";v
            ="152"',
        'sec-ch-ua-mobile': '?0',
        'sec-ch-ua-platform': '"Windows"',
        'sec-fetch-dest': 'empty',
        'sec-fetch-mode': 'cors',
        'sec-fetch-site': 'cross-site',
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
            KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36'
    }

    try:
        response = requests.request("GET", url, headers=headers, data=payload)
    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch teams for season {season}: {e}")
        return []

    return parse_teams_api(response)


def parse_teams_api(response):
    """
    parse the teams API response of one season
    return a list of team codes, e.g. IST
    """
    data = json.loads(response.text)
    team_list = []
    for team in data["data"]:
        team_list.append(team["code"])

    return team_list


def request_roster_api(team, season):
    """
    team: team code, e.g. IST
```

```
    season: season year, e.g. 2021 for season 2021-22
    returns a dictionary of player codes corresponding to their positions
    """
    url = f"https://feeds.incrowdsports.com/provider/euroleague-feeds/v2/competitions
        /E/seasons/E{season}/clubs/{team}/people"

    payload = {}
    headers = {
        'accept': 'application/json, text/plain, */*',
        'dnt': '1',
        'accept-language': 'fi-FI,fi;q=0.9,en-US;q=0.8,en;q=0.7',
        'origin': 'https://www.euroleaguebasketball.net',
        'priority': 'u=1, i',
        'referer': 'https://www.euroleaguebasketball.net/',
        'sec-ch-ua': '"Chromium";v="152", "Not?A_Brand";v="24", "Google Chrome";v
            ="152"',
        'sec-ch-ua-mobile': '?0',
        'sec-ch-ua-platform': '"Windows"',
        'sec-fetch-dest': 'empty',
        'sec-fetch-mode': 'cors',
        'sec-fetch-site': 'cross-site',
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
            KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36'
    }

    try:
        response = requests.request("GET", url, headers=headers, data=payload)
    except requests.exceptions.RequestException as e:
        print(f"Failed roster fetch for team {team}, season {season}: {e}")
        return {}

    return parse_roster_api(response)


def parse_roster_api(response):
    """
    parse the roster API response
    returns a dictionary of player codes corresponding to their positions
    """
    player_positions = {} # player codes as keys and positions as values, e.g.
        {013422 : 1}
                          # (where 1=guard, 2=forward, 3=center)
    data = json.loads(response.text)
    for player in data:
        player_positions[player["person"]["code"]] = player["position"]
    return player_positions


def request_players_api(season):
    """
    returns a list of player dictionaries
    """
    url = f"https://feeds.incrowdsports.com/provider/euroleague-feeds/v3/competitions
        /E/statistics/players/traditional?seasonMode=Single&limit=1000&sortDirection=
        ascending&seasonCode=E{season}&statisticMode=perGame&statisticSortMode=perGame
        "

    payload = {}
```

```
    headers = {
        'sec-ch-ua-platform': 'Windows',
        'Referer': 'https://www.euroleaguebasketball.net/',
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
            KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36',
        'Accept': 'application/json, text/plain, */*',
        'sec-ch-ua': '"Chromium";v="152", "Not?A_Brand";v="24", "Google Chrome";v
            ="152"',
        'sec-ch-ua-mobile': '?0'
    }

    try:
        response = requests.request("GET", url, headers=headers, data=payload)
    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch player stats for season {season}: {e}")
        return []

    return parse_players_api(response)


def parse_players_api(response):
    """parse the players API response"""
    data = json.loads(response.text)
    return data["players"]


def scrape_data():
    player_stats = []  # the final dataset -- a list of player dictionaries containing
        stats
    teams_dict = {}     # lists the teams in a given season, e.g. key "2021"
        corresponds to
                        # the team codes in the season 2021-22
    seasons = range(2021, 2026, 1) # From 2021-2022 to 2025-2026

    # Which fields to keep in player stats (other fields are dropped)
    keep = ["gamesPlayed", "gamesStarted", "minutesPlayed",
            "twoPointersMade", "twoPointersAttempted",
            "threePointersMade", "threePointersAttempted", "offensiveRebounds", "
                defensiveRebounds",
            "totalRebounds", "assists", "steals", "turnovers", "blocks", "
                freeThrowsMade",
            "freeThrowsAttempted", "foulsDrawn"]

    for season in seasons:
        roster_positions = {} # dictionary of player codes corresponding to their
            positions;
                              # season-specific
        teams_dict[season] = request_teams_api(season)

        for team_code in teams_dict[season]:
            roster_positions.update(request_roster_api(team_code, season))

        raw_player_stats = request_players_api(season)
        for player in raw_player_stats:
            row = {k: player[k] for k in keep}
            row["code"] = player["player"]["code"]
            row["name"] = player["player"]["name"]
            row["season"] = season
```

```
            row["position"] = roster_positions.get(row["code"])
            player_stats.append(row)

    return player_stats

data = scrape_data()
df = pd.DataFrame(data)
df.to_csv("data.csv", index=False)
```

B. Feature Engineering

1_feature_engineering

October 6, 2026

In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
```

In [2]:

```
df_raw = pd.read_csv("data/raw_data.csv")

df_raw.head()
```

Out[2]:

```
   gamesPlayed gamesStarted minutesPlayed twoPointersMade \
0         26.0          0.0      8.480128            0.1
1         31.0          3.0      6.978495            0.2
2         25.0         20.0     10.805333            0.4
3         23.0          2.0      7.260870            0.7
4         26.0          3.0     12.396154            0.6

   twoPointersAttempted threePointersMade threePointersAttempted \
0                   0.4              0.1                   0.9
1                   0.5              0.8                   2.2
2                   0.8              0.6                   1.4
3                   1.4              0.1                   0.5
4                   1.5              0.3                   1.4

   offensiveRebounds defensiveRebounds totalRebounds ... steals \
0                0.2               0.3           0.5 ...    0.2
1                0.0               0.4           0.4 ...    0.1
2                0.2               0.6           0.7 ...    0.4
3                0.2               0.5           0.7 ...    0.3
4                0.2               0.4           0.7 ...    0.3

   turnovers blocks freeThrowsMade freeThrowsAttempted foulsDrawn \
0        0.4    0.1            0.3                 0.4        0.4
1        0.2    0.0            0.0                 0.1        0.4
2        0.3    0.1            0.0                 0.0        0.1
3        0.0    0.2            0.3                 0.6        0.6
4        0.1    0.6            0.6                 0.7        0.8

     code            name  season  position
0  007265   KURUCS, ARTURS    2021         1
1  004508    OBST, ANDREAS    2021         1
2  007338 UZINSKII, DMITRII    2021         2
3  011220       ZIV, IFTAH    2021         1
4  011204    GRANT, JERIAN    2021         1

[5 rows x 21 columns]
```

B.1. Preprocessing

First we drop the rows with very little games (<10 per season) or minutes (<10 per game) played. These rows belong to coaches, for example.

In [3]:

```
df_filtered = df_raw[(df_raw["minutesPlayed"] >= 10) & (df_raw["gamesPlayed"] >= 10)
    ].reset_index()
print(f"Dropped {len(df_raw) - len(df_filtered)} rows with not enough games/minutes
    played. Now {len(df_filtered)}.")
```

Out[3]:

```
Dropped 46 rows with not enough games/minutes played. Now 927.
```

Next we compute the features from the raw statistics and put them into a new dataframe. Some features require median imputation as they include values such as `nan` and `inf`.

In [4]:

```
df = pd.DataFrame()

df["POS"] = df_filtered["position"]
df["code"] = df_filtered["code"]
df["3PAr"] = df_filtered["threePointersAttempted"] / (
    df_filtered["twoPointersAttempted"] +
    df_filtered["threePointersAttempted"] )
df["3Pr"] = df_filtered["threePointersMade"] / df_filtered["threePointersAttempted"]
df["2Pr"] = df_filtered["twoPointersMade"] / df_filtered["twoPointersAttempted"]
df["2P"] = df_filtered["twoPointersMade"] / df_filtered["minutesPlayed"] * 40
df["FD"] = df_filtered["foulsDrawn"] / df_filtered["minutesPlayed"] * 40
df["FTr"] = df_filtered["freeThrowsAttempted"] / (
    df_filtered["twoPointersAttempted"] +
    df_filtered["threePointersAttempted"] )
df["AST"] = df_filtered["assists"] / df_filtered["minutesPlayed"] * 40
df["TR"] = df_filtered["totalRebounds"] / df_filtered["minutesPlayed"] * 40
df["BLK"] = df_filtered["blocks"] / df_filtered["minutesPlayed"] * 40
df["AST/TO"] = df_filtered["assists"] / ( df_filtered["turnovers"] )

# convert EuroLeague's numerical labels into more interpretable ones
df["POS"] = df["POS"].map({1: "G", 2: "F", 3: "C"})

# replace AST/TO's infinite values with NaN and perform median imputation
df["AST/TO"] = df["AST/TO"].replace([np.inf, -np.inf], np.nan)
df["AST/TO"] = SimpleImputer(missing_values=np.nan, strategy='median').fit_transform(
    df[["AST/TO"]])

# perform median imputation for 3Pr as well since it has many NaNs
df["3Pr"] = SimpleImputer(missing_values=np.nan, strategy='median').fit_transform(df
    [["3Pr"]])

df.round(3).head()
```

Out[4]:

	POS	code	3PAr	3Pr	2Pr	2P	FD	FTr	AST	TR	BLK
0	F	007338	0.636	0.429	0.500	1.481	0.370	0.000	1.111	2.591	0.370
1	G	011204	0.483	0.214	0.400	1.936	2.581	0.241	2.259	2.259	0.323
2	F	004761	0.750	0.333	0.333	0.900	1.499	0.028	1.799	2.699	0.000
3	F	006251	0.613	0.211	0.500	2.038	1.699	0.129	0.679	7.474	0.000

```
4   G  002969 0.667 0.286 0.571 2.983 1.119 0.119 1.119 1.119 0.000

   AST/TO
0   1.000
1   1.167
2   3.000
3   0.400
4   1.000
```

B.2. Pearson Correlation Heatmap

To examine the correlations between features and possibly find redundant features, we make a Pearson correlation matrix and visualize it as a heatmap.

In [5]:

```
corr_matrix = df.drop(["POS", "code"], axis=1).corr() # make the correlation matrix

# visualize the matrix as a heatmap
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Pearson Correlation Heatmap")
plt.show()
```

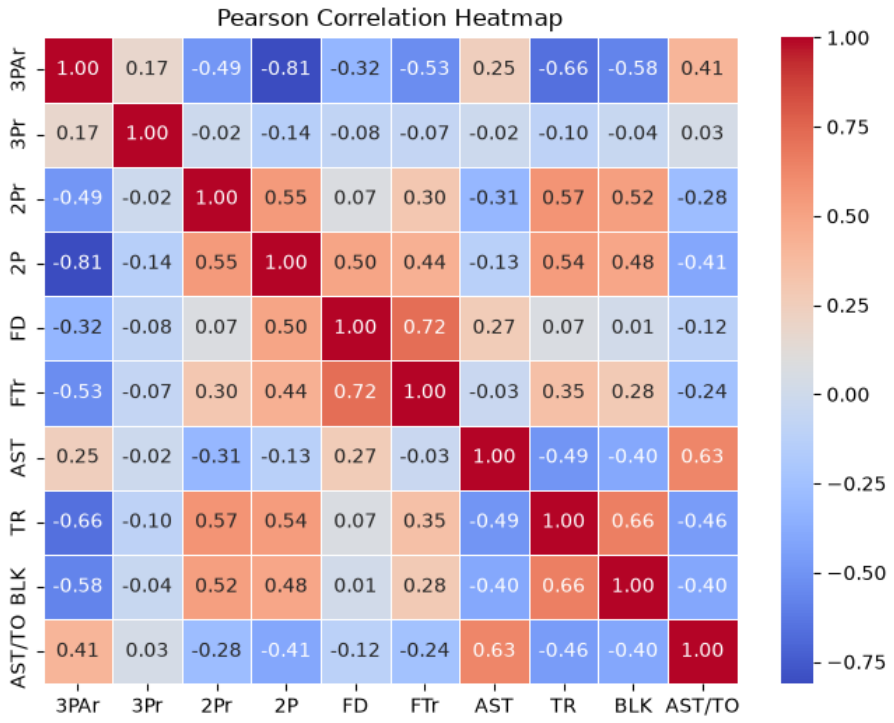


Figure 1: Pearson Correlation Heatmap

In [6]:

```
# We drop FD due to the high correlation but keep 2P and 3PAr due to the
# difference in what they measure.
df = df.drop(columns=["FD"])
```

B.3. Exploratory Data Analysis

Exploring the data. What does the data look like, and is there something that we should consider later on?

In [7]:

```
df.describe().round(2)
```

Out[7]:

	3PAr	3Pr	2Pr	2P	FTr	AST	TR	BLK	AST/TO
count	927.00	927.00	927.00	927.00	927.00	927.00	927.00	927.00	927.00
mean	0.40	0.35	0.54	3.97	0.29	3.40	6.10	0.49	1.53
std	0.21	0.10	0.09	1.85	0.14	2.29	2.48	0.59	0.80
min	0.00	0.00	0.27	0.26	0.00	0.00	1.12	0.00	0.00
25%	0.28	0.32	0.49	2.59	0.19	1.70	4.09	0.13	1.00
50%	0.44	0.36	0.54	3.78	0.27	2.61	5.71	0.29	1.40
75%	0.55	0.39	0.60	5.05	0.37	4.56	7.82	0.67	2.00
max	0.93	1.00	0.89	10.04	0.96	12.31	13.89	3.52	7.50

What's noteworthy:

- total rebounds (TR) has a wide range of values (1.12 - 13.89) so it might be informative
- blocks (BLK) are heavily right-skewed, so maybe only a tiny chunk of players get lots of blocks

Let's look at the distributions of some interesting features and see how they relate to positions.

In [8]:

```
fig, axes = plt.subplots(2, 2, figsize=(8, 8))
axes = axes.flatten()
order = ["G", "F", "C"]

sns.boxplot(data=df, x="POS", y="TR", order=order, ax=axes[0])
axes[0].set_title("TR by Position")

sns.boxplot(data=df, x="POS", y="3PAr", order=order, ax=axes[1])
axes[1].set_title("3PAr by Position")

sns.boxplot(data=df, x="POS", y="BLK", order=order, ax=axes[2])
axes[2].set_title("BLK by Position")

sns.boxplot(data=df, x="POS", y="3Pr", order=order, ax=axes[3])
axes[3].set_title("3Pr by Position")

plt.tight_layout()
plt.show()
```

- total rebounds (TR) seems to be a solid overall differentiator
- 3-pointer attempt rate (3PAr) seems to separate centers well
- the right-skewedness of blocks (BLK) is explained by mostly only centers having high values
- 3-pointer shooting accuracy seems to be similar for all positions

In [9]:

```
df["POS"].value_counts()
```

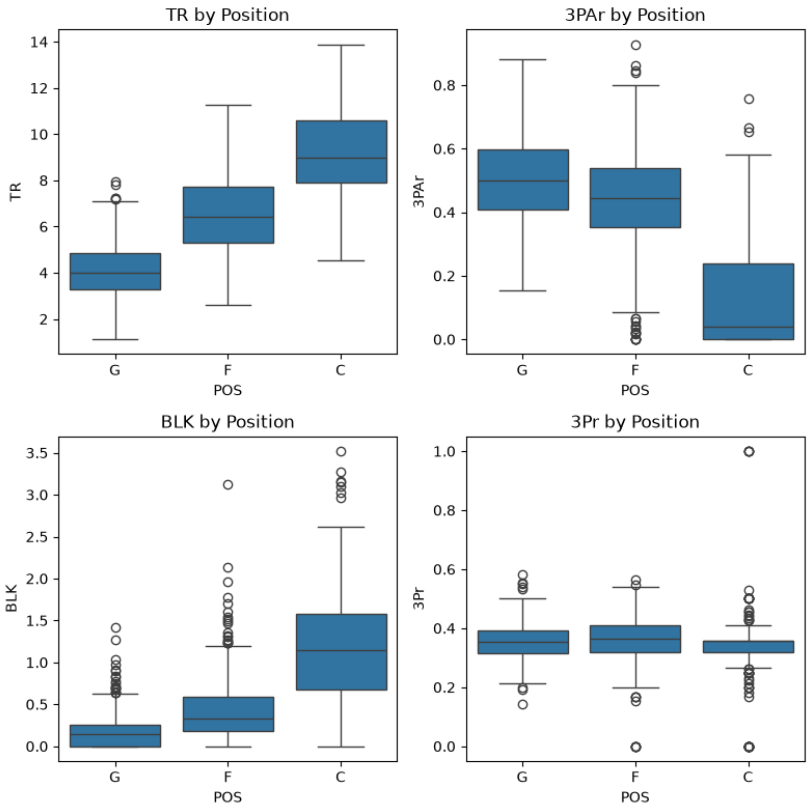


Figure 2: Feature distributions by position: total rebounds, 3P attempt rate, blocks, and 3P shooting accuracy

Out[9]:

```
POS
G    379
F    354
C    194
Name: count, dtype: int64
```

The classes are imbalanced as there are almost twice as many guards and forwards than centers. This needs to be taken later on into account when splitting the data and training the models.

In [10]:

```
# write the processed DataFrame to csv
df.to_csv("data/processed_data.csv", index=False)
```

C. Modeling and Evaluation

2_modeling

October 6, 2026

In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, StratifiedGroupKFold,
    GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, f1_score
```

In [2]:

```
df = pd.read_csv("data/processed_data.csv")
```

C.1. Splitting the Data

First, we split the data into player codes, features X, and labels y. Player codes are needed to group by player in the splitting process.

In [3]:

```
player_codes = df["code"]
y = df["POS"]
X = df.drop(["POS", "code"], axis=1)
```

Now let's split the data further into an 80 % training set and a 20 % test set. This method keeps the class proportions the same as in the original dataset and groups by player so the same player's different seasons do not end up on both the training and the test set.

In [4]:

```
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=1)
train_idx, test_idx = next(sgkf.split(X, y, groups=player_codes))

X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
codes_train, codes_test = player_codes.iloc[train_idx], player_codes.iloc[test_idx]
```

In [5]:

```
# defines which split strategy the 5-fold cross-validations are going to use later on
cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=1)
```

C.2. Training

As classifiers we use random forest and multinomial logistic regression. In both cases, model training and parameter tuning uses 5-fold cross-validation on the training set. First, we perform a wide search to find the general ballpark of the parameters, then a narrower search centered on that region to find the best values more precisely.

Macro-averaged F1 score is used as the evaluation metric since the dataset is imbalanced with equally important classes. Both classifiers also use balanced class weights for the same reason.

C.2.1. Random Forest

In [6]:

```
# which parameters to try
param_grid = {"max_depth": [3, 4, 5, 6, 7, 8, 10, None], "n_estimators": [100, 200,
    300, 400, 500]}

forest = RandomForestClassifier(class_weight="balanced", random_state=1)

# GridSearchCV performs the cross-validation and trains the model
rf_grid = GridSearchCV(forest, param_grid=param_grid, cv=cv, scoring="f1_macro")
rf_grid.fit(X_train, y_train, groups=codes_train)

print("best RF params: ", rf_grid.best_params_, "\nbest macro F1: ", rf_grid.
    best_score_)
```

Out[6]:

```
best RF params: {'max_depth': 3, 'n_estimators': 300}
best macro F1: 0.786774272566816
```

Now let's narrow down the parameters.

In [7]:

```
param_grid = {"max_depth": [1, 2, 3, 4], "n_estimators": [250, 275, 300, 325, 350,
    375]}

# overwrite the earlier model
rf_grid = GridSearchCV(forest, param_grid=param_grid, cv=cv, scoring="f1_macro")
rf_grid.fit(X_train, y_train, groups=codes_train)

print("best RF params: ", rf_grid.best_params_, "\nbest macro F1: ", rf_grid.
    best_score_)
```

Out[7]:

```
best RF params: {'max_depth': 3, 'n_estimators': 300}
best macro F1: 0.786774272566816
```

In [8]:

```
rf_model = rf_grid.best_estimator_ # Save the trained model as rf_model

# Print the training macro-F1 score
print("RF training macro F1: ", f1_score(y_train, rf_model.predict(X_train), average=
    "macro"))
```

Out[8]:

```
RF training macro F1: 0.8204414247854638
```

With params figured out and the model trained (`GridSearchCV` trains the model as well), we move on and train the multinomial logistic regression model.

C.2.2. Multinomial Logistic Regression

Logistic regression requires standardized features. We use `Pipeline` to standardize the data in each CV fold separately based on only the training set's values, so that none of the scaling depends on the validation set's values.

In [9]:

```
lr_pipe = Pipeline([('scaler', StandardScaler()),
                    ('clf', LogisticRegression(class_weight='balanced', max_iter=1000,
                         random_state=1))])

lr_param_grid = {"clf__C": [0.01, 0.1, 1, 10, 100]}
lr_grid = GridSearchCV(lr_pipe, param_grid=lr_param_grid, cv=cv, scoring="f1_macro")
lr_grid.fit(X_train, y_train, groups=codes_train)

print("best LR params: ", lr_grid.best_params_)
print("best LR macro F1: ", lr_grid.best_score_)
```

Out[9]:

```
best LR params: {'clf__C': 0.01}
best LR macro F1: 0.7917975232533492
```

Narrow down the parameters

In [10]:

```
lr_pipe = Pipeline([('scaler', StandardScaler()),
                    ('clf', LogisticRegression(class_weight='balanced', max_iter=1000,
                         random_state=1))])

lr_param_grid = {"clf__C": [0.001, 0.003, 0.005, 0.007, 0.01, 0.02, 0.03, 0.05]}
lr_grid = GridSearchCV(lr_pipe, param_grid=lr_param_grid, cv=cv, scoring="f1_macro")
lr_grid.fit(X_train, y_train, groups=codes_train)

print("best LR params: ", lr_grid.best_params_)
print("best LR cross-validation macro F1: ", lr_grid.best_score_)
```

Out[10]:

```
best LR params: {'clf__C': 0.01}
best LR cross-validation macro F1: 0.7917975232533492
```

In [11]:

```
# Save the trained logistic regression model as lr_model
lr_model = lr_grid.best_estimator_

# Print the training macro-F1 score
print("LR training macro F1: ", f1_score(y_train, lr_model.predict(X_train), average=
    "macro"))
```

Out[11]:

```
LR training macro F1: 0.7943718215671033
```

C.3. Evaluation

With parameters figured out, we evaluate the models' performance on the test set.

In [12]:

```
y_pred_rf = rf_model.predict(X_test)
y_pred_lr = lr_model.predict(X_test)
```

C.3.1. Random Forest Model

Performance metrics: confusion matrix, classification report, and feature importances.

Confusion Matrix

In [13]:

```
name_map = {'G': 'Guard', 'F': 'Forward', 'C': 'Center'} # to correctly label the
    classes in the figures

# get and reshape confusion matrix data
rf_confusion_matrix = confusion_matrix(y_test, y_pred_rf, labels=rf_model.classes_)
rf_confusion_matrix = rf_confusion_matrix.astype('float') / rf_confusion_matrix.sum(
    axis=1)[:, np.newaxis]

# make the heatmap
plt.figure(figsize=(6,3))
sns.set(font_scale=1)
sns.heatmap(rf_confusion_matrix, annot=True, annot_kws={'size':10}, cmap=plt.cm.
    Greens, linewidths=0.2)

# add labels
labels = [name_map[c] for c in rf_model.classes_]
tick_marks = np.arange(len(labels)) + 0.5
tick_marks2 = tick_marks
plt.xticks(tick_marks, labels, rotation=0)
plt.yticks(tick_marks2, labels, rotation=0)
plt.xlabel('Predicted label')
plt.ylabel('True label')
plt.title('Confusion Matrix for Random Forest Model')
plt.show()
```

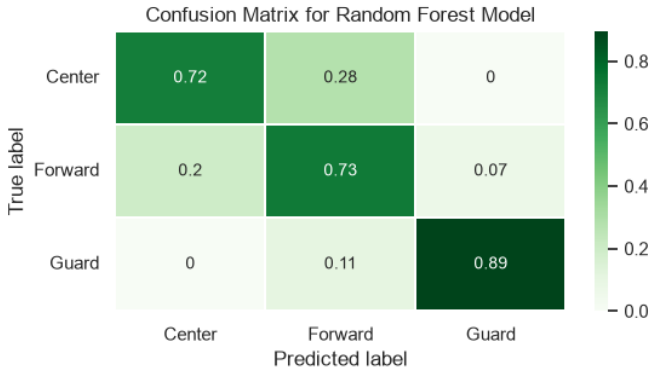


Figure 3: Confusion matrix for the random forest model (row-normalized)

Classification Report

In [14]:

```
rf_report = classification_report(y_test, y_pred_rf, output_dict=True)
rf_report_df = pd.DataFrame(rf_report).transpose().round(3)
rf_report_df.to_csv("visualizations/rf_classification_report.csv")
rf_report_df
```

Out[14]:

	precision	recall	f1-score	support
C	0.667	0.718	0.691	39.000
F	0.732	0.732	0.732	71.000
G	0.932	0.895	0.913	76.000
accuracy	0.796	0.796	0.796	0.796
macro avg	0.777	0.782	0.779	186.000
weighted avg	0.800	0.796	0.797	186.000

What's especially noteworthy is that the test macro F1 (0.779) is close to the cross-validation macro F1 (0.787), suggesting that the model generalizes well.

Feature Importances

Next, let's see how much each feature contributed in the classification.

In [15]:

```
importances = pd.Series(rf_model.feature_importances_, index=X_train.columns, name="
    importance")
importances = importances.sort_values(ascending=False)

importances_df = importances.reset_index().rename(columns={"index": "feature"}).round
    (3)
importances_df.to_csv("visualizations/rf_feature_importances.csv", index=False)
importances_df
```

Out[15]:

feature	importance
TR	0.319
3PAr	0.265
AST	0.172
BLK	0.100
2P	0.056
2Pr	0.038
AST/TO	0.033
FTr	0.013
3Pr	0.005

C.3.2. Multinomial Logistic Regression

What logistic regression gives us that random forest doesn't is signed coefficients that tell us detailed information about the interactions between features and labels.

Coefficients

In [16]:

```
lr_model = lr_grid.best_estimator_.named_steps["clf"]
coef_df = pd.DataFrame(lr_model.coef_, columns=X_train.columns, index=lr_model.
    classes_).round(2)

plt.figure(figsize=(10,3))
sns.set(font_scale=1)
sns.heatmap(coef_df, annot=True, annot_kws={'size':10}, cmap=plt.cm.Greens,
    linewidths=0.2)

# add labels
labels = [name_map[c] for c in coef_df.index]
tick_marks2 = tick_marks
plt.yticks(tick_marks2, labels, rotation=0)
plt.title("Multinomial Logistic Regression Coefficients by Position")
plt.show()
```

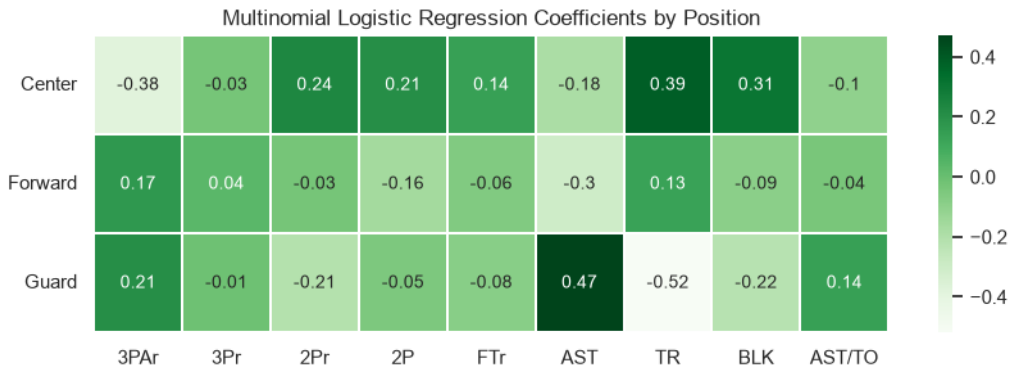


Figure 4: Multinomial logistic regression coefficients by position

Confusion Matrix

In [17]:

```
# get and reshape confusion matrix data
lr_confusion_matrix = confusion_matrix(y_test, y_pred_lr, labels=lr_model.classes_)
lr_confusion_matrix = lr_confusion_matrix.astype('float') / lr_confusion_matrix.sum(
    axis=1)[:, np.newaxis]

# make the heatmap
plt.figure(figsize=(6,3))
sns.set(font_scale=1)
sns.heatmap(lr_confusion_matrix.round(2), annot=True, annot_kws={'size':10}, cmap=plt
    .cm.Greens, linewidths=0.2)

# add labels
labels = [name_map[c] for c in lr_model.classes_]
tick_marks = np.arange(len(labels)) + 0.5
tick_marks2 = tick_marks
plt.xticks(tick_marks, labels, rotation=0)
plt.yticks(tick_marks2, labels, rotation=0)
plt.xlabel('Predicted label')
plt.ylabel('True label')
plt.title('Confusion Matrix for Multinomial Logistic Regression')
plt.show()
```

Classification Report

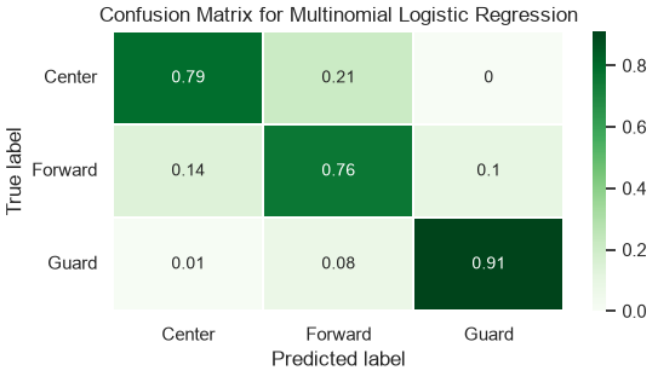


Figure 5: Confusion matrix for the multinomial logistic regression model (row-normalized)

In [18]:

```
lr_report = classification_report(y_test, y_pred_lr, output_dict=True)
lr_report_df = pd.DataFrame(lr_report).transpose().round(3)
lr_report_df.to_csv("visualizations/lr_classification_report.csv")
lr_report_df
```

Out[18]:

	precision	recall	f1-score	support
C	0.738	0.795	0.765	39.000
F	0.794	0.761	0.777	71.000
G	0.908	0.908	0.908	76.000
accuracy	0.828	0.828	0.828	0.828
macro avg	0.813	0.821	0.817	186.000
weighted avg	0.829	0.828	0.828	186.000

For logistic regression, test macro-F1 (0.817) is actually *higher* than in cross-validation (0.792) or training (0.794).